Assignment #2

전기전자공학부 202110939 신주엽

```c
#include <stdio.h>
#include <stdlib.h>
#define W 3 // any odd number

void read_file(int* input, int height, int width, char filename[])
{
    int x, y;
    FILE* fp;
    fp = fopen(filename, "r");
    for (y = 0; y < height; y++) {
        for (x = 0; x < width; x++) {
            fscanf(fp, "%d", &input[y * width + x]);
        }
        fscanf(fp, "₩n");
    }
    fclose(fp);
}

void insertion_sort(int* a, int N)
{
    int i, j, t;
    for (i = 1; i < N; i++)
    {
        t = a[i];
        j = i;
        while (j > 0 && a[j - 1] > t)
        {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = t;
    }
}
```

```c
void median_filtering(int *input, int *output, int height, int width)
{
    int half = W / 2;
    int window[W * W];

    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (i < half || i >= height - half || j < half || j >= width - half) {
                output[i * width + j] = input[i * width + j]; // Out of Boundary 처리(현재 값 그대로 적용)
                continue;
            }
            for (int y = 0; y < W; y++) {
                for (int x = 0; x < W; x++) {
                    window[y * W + x] = input[(i + y - half) * width + (j + x - half)];
                }
            }
            insertion_sort(window, W * W);
            output[i * width + j] = window[W * W / 2]; // 중앙 값으로 대체
        }
    }
}
```

```c
void main()
{
    int height = 44, width = 45; // image width and height
    int* source;
    int* result;

    source = malloc(sizeof(int) * height * width);
    result = malloc(sizeof(int) * height * width);

    read_file(source, height, width,
        "C:₩₩Users₩₩shinj₩₩Desktop₩₩3-1₩₩EmbeddedDataStructure₩₩noisy_data.txt");

    median_filtering(source, result, height, width);

    printf("Source₩n");
    for (int i = 0; i < height * width; i++) {
        if (i % width == 0) printf("₩n");
        printf("%3d ", source[i]);
    }
    printf("₩n₩n");

    printf("Result₩n");
    for (int i = 0; i < height * width; i++) {
        if (i % width == 0) printf("₩n");
        printf("%3d ", result[i]);
    }
    printf("₩n₩n");

    free(source);
    free(result);
}
```

Microsoft Visual Studio 디버그

Source

```
  0  0  2  0  2 12  0 54  0  0  0 18  0  0  2 42 25  0  0  0  0 13  0 14 49  0  0 52  0  0  0  1 22 13  0 19  0  0  0  0  0  2  0  0  8
  0  0 31  0  0  0 19  1  0  0  0 23 48  0  0 27 10  0  0  8  4  9 14  0  0  0  0 20  8  4  0  0 11 18 49  0  0  0  0  0 22 40  0 14
 23  0 31  0 34  4  9  0 16 51  0  0  3  0 60  0  0  0 17  0 11  0 41  0  0 42 10 57 12  0  0  0  0 17  0 24  0  0  0 74  0  1 24 15  8
 52  0  0  0  0 19  3  0  6 44  0  0 42  0 35  0 24  0  0 20 12  3  0  0  0  0 23  0  0 34  0  0  0  0  0  0  0  7  0  2 28 26
  9  8  4  3  0  9  9  0  0 33  0  0  0  0  0  0  0  0  5 22  0 45 25  0  0  0 17 12 25  0  0  0  0  0 56 31 13  8 16  6  0 36  0  0  0
  9  0  0  0 24  0 22 18  0  0  0  0  0 16 18  0  0 14  0 22  0  0  0 233 254 236 255  22  0 11  0  0 16  0  0  4  0 31  0 11 24  0  5  0  0
  8  0 52  6  0  0  0  0 10  0  0 53  0  0 28 49  0  0 30 10  0  0 255 255 240 255 240 215 217 10 22 30  5  6 21 52  5 30 41  4  8  0  0 20  0
  0  0  0  0  0 16 25  0  6 29 28  9  0  0  0 26 13 43  0 16 20 195 222 219 255 255 192 250 221 213  0  0 24 22  0  0 18 16  0  0  3 12 15  0 12
 15  0  0  0  0  0  1 12 11  0 15 15 48  0 19  0  0  0 55  9  0 210 234 254 249 249 247 255 237 255 255  0  3 14 21  0  0  3 16  0  0 11  0  0 16
 15  0  0 25 50 12  0  0  5  0  0 25 22  7 18  5 33  0 23 255 203 255 223 233 255  0 226 255 255 255  0 10  0 24  7  0  0  0  3 15  0 16 45  0  0
  0  0  0  0  0 33  0 29  0 25 16 21 40 13  6  0 12 34  6 255 244 239 238 255 220 35 255 255 255 255 231 30 51  0 46  0 21  0  0 12 59  0  0  0  6
  8  1 16  6  0 15 24  0  0  5 11  0  9  0  8 57  0  0 19  0 199 224 226 247  0  8  3  0 211 255 246 255  0  0  7  5  0 33  0  0  0  0 40  0  0
 23  2 13  0  0  0 10  0  0  2  0  0 15  7 12  1  0  0  0 248 247 255 239  0 13  1  0 255 246 234 253 253 25  0  0  0  0  0  0  0 43  0  1 30  0
  0 21 30  0  0 56  2 35  9 36  0  4 20  0 23 28  0  0  0 255 255 243 255  0  0 41 62 37 255 237 230 252  0  0 61  4  0  0 33 28 40  0  8  0  0
  0 42 26  0  0  0 31  0  0  0  0 36  1 15 14 39 23  0 255 255 255 215 230  1  0  5  0 11 244 255 255 241  0 39 43 22  0  0  0  0  3  0  0  0
 32 39  0 22 29  0 41  0  0  6  0 10  9  3  0 29 250 255 254 216 255 255  0  0  0  0  9 238 255 255 255 255 33  9  8 42  0 10  0  0 23  6 44  7
  0  0  0  0  0 14  0  0 23  5  0  0  8 16 46  0 238 255 255 233 255  0  0 21  0  3  4 12  0 255 255 226 255  0 43  0 20  0 42  0  5 23  0 18  0
 23  0  0  0 10  0  2  0 33 14  0  3  6 24  0  0 255 234 236 237 43 35  0 19 33  0 33  0  3 255 182 221 255  0  0  6 27  4 14  0 71  0  0  0 68
 11  0  0  0  0 24  9  0  0 17  0  0  7  1  0 255 255 255 255 255  0  0  0 18 17 28 10  0 28 210 255 255 243 199  0 34  0  0 23  0  0  3  8 49  0
  0  0 14  0  0  0  0 47 38 21  0  0  0 11 24 235 241 238 255  0 16  0  0 38  0 62 16  0  0 255 210 255 255  0  0  0  0 12  0  0  0  7 18  0
  6  4  0  8 55  9  0  0  5  3  9  0  3 15  0 248 255 255 216 16  0 17  0 10  0 27  2 11  4  1 255 251 255 255  0 32  0  7  0 19  0  0  0 56 44
  0  0 15  0  6 40 18  0  6  4  0  0  1  0 226 255 255 240 255 17 18  0  0 17 35  7 14  0 13  0 238 255 255 247 225  0 17  0  0 11 10  0  0  5  0
 39 15  0  0 15 14  0  0  0 17  0 18  0  0 236 233 255 255  0  0 18  9 18  5 51  8  0  0  0 64  0  0 193 246 229 255 245 12  0  0 28  0 22 31 29  0
 16 17  0  5 15 36  0  0  0  0 17 14 48 255 255 255 232 255 235 28 26 31  0  0 11  5  0 243 250 255 248 255 250 255 255 255 14  0 12 24 37 15 27  0 48
 50  0  0 12 24  0 39  4  9  0 52 42 239 248 255 226 248 251 255 255 255 255 255 255 255 255 230 255 255 255 253 250 255 193 232 41 10  0  0  0 26 31  0 39
  3  0  6 18  4  0 12  0  0  0 251 239 253 238 237 231 255 222 233 254 255 249 234 247 255 255 251 255 250 231 226 255 255 247 238 27  0  0 84 17 15  0  0 16
  0 30 16  0  0  0 26 35 16  0 217 200 255 255 224 255 246 240 255 255 237 242 255 255 255 255 249 245 212 255 253 255 255 243 232 221  0  0  0  0  0  2  0  2
  0 13  1  0  0 60  0  0  0 35 255 255 255 245  0 14 15 63 24 251 239 231 245 223 241 255 255 223  1  0  0  0 228 238 239 255 228  0  0 11  8  0 39  0 14
  0 34  0  0  6  0  0  0  5 230 255 255 255 255 40  0 29 15 46  2  7  0  4 11  0  0  0 28  0  0  0  0 63 244 243 250 198 250  0 16  0  0 20  0 23
 19  0  0  0 39  0 13  0 10 238 231 255 255  0  0  3 67  2  4  0  0 11  0 10  0  6  0  8 10  0  0  0 24 18 215 242 255 255 230 255  3  0 14  0  0 15
  0  0 50  0  0  0  1  0 229 255 255 255 255  0  0  0  0  0 17  0 39 17  0  6 16 15  0  0  1  0  4  0  1  7 255 255 254 207 242 23 22 15  0  0  3
  0 47 18  0  7  0  0 10 250 255 202 221 32  0  0 19  0  0  0 37 15 17  0  1 16  0  0 36  0 39  2 13 31 14  0 255 255 255 255 42  0  2 23  0  0
  0 8 34  5  0  2  0  0 255 255 255  0 28  0  0 56  2 24  0  0  1  1 31 15  0  7  0  0  0  0  8 11 11  0 226 227 246 255 24  0  0 58  0  0
  0  0  0 23 48 27  9  0 255 255 236 236  0  0 28  4  0  0  0 37  0  4  0 64 22 20  1  4 19  0  0  0  0 17 18 255 255 255 234 255  0  0  0 14  3
  0  5  0  0  0 12  0 246 255 250 255 255  0 15  0  6  8  0  0 11  0  3 11  0  0  0  0 27 15  0 23  0 21  0  0 36 245 232 255 255 38  0  0 37  2
  0  0  0 48  0  0  0 255 255 255 255 42 36  7  1 27  0 69 19  0 28  0  0  8  0 19  0  9  0  0 40  0  7  0 24  0 246 255 255 246  0 16  0  0 38
  0 24  0  2 39  0  2 255 255 255  0  0  0 50  5  0 84 44 28  0 17  0  0 10  0 10  0  0 24 49  0  0 21  0  0  0 20 255 255 255  0 33  0 30  0
  0  0  0  5  0 24  0 41 255 10  0  8 14  0  0  0 35  2 13  6  0  5  0 26  0 14  8 32  0 28  0  0 52 30 17  0 24 49 43 27  0  6  0 18  0
 16 61 28  0  0  0 28  0  3 13 15  0 13 14 59 18  0  0  0  0  0 37 42 14  7 32 21  0  0  0  0  0  0 17  7  0  7  1 16  5  3 14  6 25  0
  9  0  0 89 54  1 14 30  0 16 27  0  0  0  0  3  0 28 10  0 24  0  6  0 50  0 10  0  9  0  0 15  0  0  0  0  0 32  0 49  7  0 10 19  3
  0 17  0 28  2  0  8 22  0  0 12 27  9 16  0  8  0  0  0 12  0  0  0  0 19  0  0 18  0  0  0  1 23 30  6  0 49  0  0 34 74  0 22 30 34
 30 12  4 29  0  0 19  0 44 10  0  0  0  0 31 18  2  0 36  8  0 39  0 29 13 12  0  0 20  7 19 18 13  0 35 14  6 21 51 13  0  0  0 36  0
 24  0  0  0  0  0  0  8  0  0  0 30  0  0  0  0  0 27 13 13  0 37 13  0  0  0 37 25  2 16  0  4  0  0  0 10  0  0 37  0  0 18 27  0 13  0
 27  0 73  0 28 39  0  5  0 20  0 55  0 13  0 15  0 24  0  0  0 11 42  0 14  0  0 13  7  0  0  0  0  0  0  0  0  0 21 43 19  0 56 14  2  7
```

Result

```
  0  0  2  0  2 12  0 54  0  0  0 18  0  0  2 42 25  0  0  0  0 13  0 14 49  0  0 52  0  0  0  1 22 13  0 19  0  0  0  0  0  2  0  0  8
  0  0  0  2  0  4  4  1  0  0  0  0  0  0  2  0  0  0  0  0  9  9  0  0  0  0  0 10  8  0  0  0  1 11 17  0  0  0  0  0  0  1  2  8 14
 23  0  0  0  0  4  3  3  1  0  0  0  0  3  0  0  0  0  0  8  4  9  0  0  0  0  0 10  0  0  0  0  0 11  0  0  0  0  0  0  2 15 15  1
 52  8  0  0  3  9  4  3  6  6  0  0  0  0  0  0  0  0  0 11 11 11  0  0  0  0 12 12 12  0  0  0  0  0 13  0  0  0  0  0  0  1  1  8 26
  9  4  0  0  0  9  9  3  0  0  0  0  0  0  0  0  0  0  5  5 12  0  0  0  0  0 17 17 11  0  0  0  0  0  0  4  0  0  6  0  2  0  0  0
  9  8  3  3  0  0  0  0  0  0  0  0  0  0  0  0  0  0 10  6  0  0 25 23 236 240 215 25 12 10  0  0  5  6 13  8 13  0 11  8  4  0  0  0
  8  0  0  0  0  0  6  0  0  0  0  0  0 16 13 13  0 14 10 10  0 219 240 254 250 248 217 213 11 10  5  6  6  4  4 16 16 11  4  4  5  0  0  0
  0  0  0  0  0  0  1  6  6 10 15  9  0  0  0  0 13  9 10 10 16 210 234 249 249 247 237 221 213 10  8 14 14  6  3  5  5  0  0  3  3  0  0 12
 15  0  0  0  9 12 11  5  5  6 15 15  9  0  5  5 13 13 16 20 203 222 233 249 249 249 247 250 255 221  8  3 14 14  0  0  0  0  0  3 11 11  0 16
 15  0  0  9  1 11  0  5  5 16 21 21 13  6  5  5 12 23 203 239 238 239 249 247 247 247 255 255 255 30 10 14 14 14  0  0  0  0  0 11  0 10  0  0
  0  0  0 12 12 12  0  5  5 11 16 13 13  7  6  5 12 23 203 239 238 238 233  35 35 211 255 255 255 231 10  7  7  5  5  0  0  0  0  0  0  0  6
  8  2  1  0  0  0 10  0  2  2  9  9  9 12  7  1  0  0 19 224 244 239 239 220  8  8 35 255 255 253 246  51  7  5  0  0  0  0  0  0  0  0  0  0
 23 13  6  3  2  0 10  5  5  2  2  0  7 12  8  1  0  0  0 224 247 247 239  0  1  3 37 211 246 246 246 230  0  0  0  0  0  0  0  0  1  0  0  0
  0 21 13  0  0  0  2  2  0  0  0  1  7 14 14 14  0  0  0 255 255 247 230  1  1  1 11 62 244 246 252 241 25  0  4  0  0  0  0  3  0  1  0  0
  0 26 22  0  0  0  2  2  0  0  0  4  9 10 14 23 23 23 255 255 255 255 230  0  0  0  9 37 238 255 255 252 39 33 22  9  0  0  0  0  3  0  3  0  0
 32  0  0  0  0  0  0  0  0  0  0  1  9 10 14 23 29 255 255 255 254 230 21  0  0  0  4  9 238 255 255 255 226 39 22 20  0  0  0  0  0  3  6  0  7
  0  0  0  0  0  0  5  5  0  3  8  9  3  3 234 255 254 237 233 43 19  0  0  0  0  3  4 12 255 255 255 243  6  9  6 14  0  0  5  5  6  6  0  0
 23  0  0  0  0  2  0  0  5  5  0  6  7  1 46 255 255 255 237 43  0  0 17 18 10  4  4 12 210 255 255 226 43  0  6  4 14  0  5  0  3  3  0 68
 11  0  0  0  0  0  2 17 14  0  0  3  6 11 235 241 255 238 236 16  0  0 17 19 17 10  3  0 182 221 255 243 199  0  0  0  4  0  0  0  0  3  7  0
  0  0  0  0  0  0  0  5  5  0  1  3 15 241 255 255 255 16  0  0  0 18 16 11  4  1 28 251 255 255 243 32  0  0  0  0  0  0  0  7  8  0
  6  0  0  6  6  6  0  5  5  5  0  0  3 24 241 248 255 238 17 16  0  0  0 17 14 11  4  0  4 238 255 255 255 32  0  0  0  0  0  0  0  0  5 44
  0  4  0  6  9 14  0  0  3  4  3  0  0 1 226 248 255 255 216 17 16  9  9 10 10  8  7  4  1  4 193 251 251 247 229 17  7  0  0  0 10  0  5  5  0
 39 15  0  5 14 15  0  0  0  0  4  1  1 48 236 255 255 255 235 18 18 18  5 11  8  8  5 13 13 64 238 248 250 250 247 225 12  0  0 11 15 15 15  5  0
 16 15  0  5 14 15  0  0  0 17 18 42 239 248 248 251 251 251 28 28 26 18 18 11 11 8 230 243 250 253 250 250 250 255 232 14 10  0  0 22 26 26 29 48
 50  3  5  6 12 12  0  0  0  0 14 48 239 248 253 238 248 248 251 235 254 254 249 247 247 255 251 251 251 255 253 250 255 255 255 232 27 10  0 12 17 17 15 16 39
  3  3  6  6  0  4  4 12  0  0 42 217 248 248 248 238 246 248 255 255 255 255 255 255 255 255 255 249 255 253 253 255 255 243 232 41  0  0  0  0  2  0  2 16
  0  3  6  1  0  0  0  0  0  0 200 251 253 245 238 231 237 231 240 239 242 245 245 247 255 255 255 251 240 231 226 231 253 247 243 238 221  0  0  0  8  2  0  2  2
  0  1  1  0  0  0  0  0  5 35 230 255 255 255 224 29 40 29 63 237 237 237 231 241 241 255 249 223 28  1  0 63 238 243 243 243 239 228  0  0  0  0  0  2 14
  0  0  0  0  0  0  0  5 230 255 255 255 245 14 14 15 15 24 15 11  7 10 10 10  6  6  8  1  0  0  0 63 238 243 243 250 228 11  3  3  0  0 14 23
 19  0  0  0  0  0  1 10 231 255 255 255 29  0  0  2  4  4  4  7  2  6  4  6  0  0  0  0  0  1 18 215 244 254 250 242 207 16 14  0  0  0 15
  0  0  0  0  0  0 10 229 238 255 221  0  0  0  0  0  0  4 15 11  6  1  6  0  0  0  1  2  4 14 18 242 255 255 255 230 23 14  2  0  0  3
  0  0  5  5  0  0  0  1 250 255 255 255 32  0  0  0  0  0  0 15 15  1  1  6 15  7  0  0  0  0  2  4 11 11 14 254 254 254 242 24 15  8  2  0  0
  0  0  5  7  5  2  0  9 255 255 255 221 28  0  0  0  0  0  0  4  0  1 15 16  7  1  1  0  0  0  2 11 11 17 227 255 255 255 242  2  0  2  0  0
  0  0  0  5  5  2  2  9 255 255 255 236 15  0  4  4  2  0  0  0  1  0  3 11 15  1  1  4  0  0  0  8 11 17 226 245 246 255 234  8  0  0  2  3
  0  0  0  0 12  9  9 246 255 255 255 236 15  1  6  4  4  0  0  0  3  0  3  0  8  0  4  4  4  0  0  0  0  7 17 36 245 255 255 246 16  0  0  2  2
  0  0  0  2  0  2 255 255 255 255 36 15  5  6  5  8 19 11 11  0  0  0  0  0  0  0  9 15  0  7  0  0  0 20 232 255 255 255 33  0  0  0 38
  0  0  0  2  5  0 255 255 255 10  8  8  5  1  1 27 28 13 13  0  0  0  0 10  0  9  0  9  0  0  0  0  0 17  0 17 24 244 255 43 16  0  6  0  0
  0  0  2  0  0  2 28 41 13  8  0  8 13  5  5  2 13  2  0  0  0 10  7 10  8 10  0  0  0  0  0  0  0 17  0  7  7 24 43 16  6  3 14  0  0
 16  0  0  0  1  1 14 14 13 13 10  8  0  0  0  0  2  2  2  0  0  5  6  7 14  8  8  0  0  0  0  0  0  7  0  0  1 16 27  7  6  6 10  6  0
  9  9 17  2  1  2  8  8  3 12 13 12  9  9  8  0  0  0  0  0  0  0  0  7  7  7  0  0  0  0  0  0  1  6  0  0  0  1  5  7  7  7 14 19  3
  0  4 12  4  2  2  8 14 10 10 10  0  0  0  3  2  2  2  0  8  8  0  0  0  6 12  0  0  0  0  0  1 13 13  6  0  6  6  6 21 13  7  0 10 19 34
 30  4  4  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 12  8  8  0  0  0 12 12  2  2  2  4  1  4  1 10  6  6  6  6 13 13 13  0 13 13  0
 24 12  0  0  0  0  0  0  5  0  0  0  0  0  0  2  2 13  0  8  0 11  0 12 12  2  7  7  7  0  0  0  0  0  0  6 21 21 13 13  0 13  2  0
 27  0 73  0 28 39  0  5  0 20  0 55  0 13  0 15  0 24  0  0  0 11 42  0 14  0  0 13  7  0  0  0  0  0  0  0  0  0 21 43 19  0 56 14  2  7
```

read_file() 함수는 텍스트 파일에서 2차원 형태의 이미지 데이터를 읽어와 1차원 배열로 저장하는 역할을 하며, height와 width를 기준으로 행렬처럼 데이터를 순차적으로 읽어 input[y * width + x] 형태로 메모리에 할당한다. 이 과정에서 파일을 fopen()으로 열고, fscanf()를 통해 숫자들을 읽어들인다.

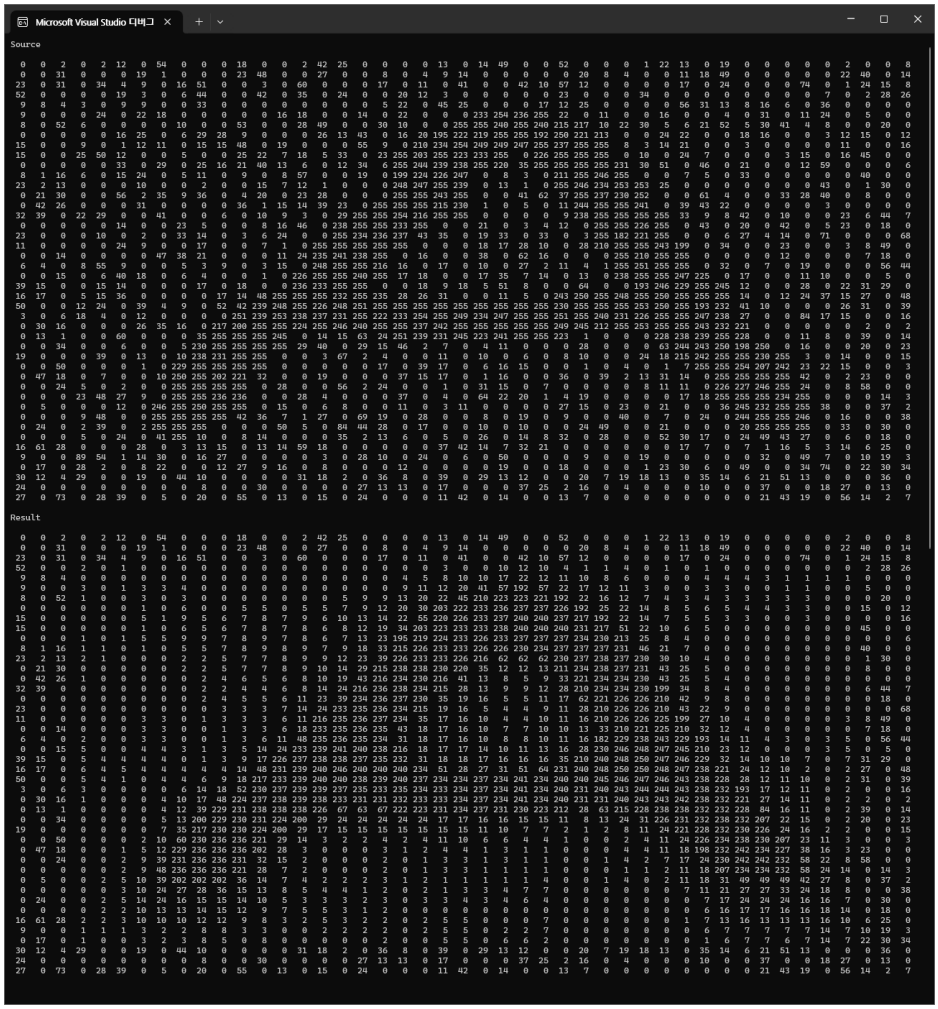
insertion_sort() 함수는 median filtering에서 사용되는 W×W 윈도우 내의 값을 정렬하기 위해 사용되며, 배열을 앞에서부터 차례로 순회하면서 앞쪽 값들과 비교하여 삽입 위치를 찾아 값을 뒤로 밀고 정렬하는 방식으로 작동한다. 데이터 수가 작기 때문에 단순한 $O(n^2)$ 정렬 알고리즘임에도 성능상 무리가 없다고 판단하여 사용하였다.

median_filtering() 함수는 각 픽셀을 중심으로 W×W 크기의 윈도우를 구성해 정렬 후 중앙값을 구해 denoising 처리를 수행한다. 이때 경계 영역(윈도우가 이미지 밖으로 넘어가는 경우)은 필터를 적용하지 않고 해당 위치의 원본 픽셀 값을 그대로 출력 배열에 복사하며, 나머지 유효한 영역에서는 window[] 배열에 주변 픽셀 값을 저장하고 insertion_sort()를 통해 정렬하고 중앙값을 결과에 대입한다.

main() 함수는 이미지 크기 변수와 입력 및 출력 배열을 동적으로 할당한 뒤 read_file() 함수를 통해 noisy 데이터를 읽고, median_filtering() 함수를 호출하여 denoising을 수행한다. 이후 원본 이미지(source)와 필터링 결과(result)를 행 기준으로 포맷에 맞게 출력하여 필터링 효과를 시각적으로 확인할 수 있도록 하며, 마지막에는 할당한 메모리를 해제한다.

코드를 실행하면 콘솔에는 먼저 원본이(Source)가 출력되고, 이어서 필터링 결과인 Result가 출력된다. 원본 이미지는 0이나 255와 같은 극단적인 값들이 군데군데 무작위로 분포되어 있으며, 그로 인해 이미지 윤곽이 뚜렷하지 않고 노이즈로 인해 형태가 깨져 보인다. 하지만 median filtering이 적용된 결과에서는 이러한 극단적인 값들이 주변의 값들로 대체되면서 전체적으로 부드러워지고, 노이즈가 줄어들며 대상의 윤곽이 더욱 뚜렷하게 보이게 된다. 특히 out-of-boundary 영역은 필터링을 적용하지 않고 원본 값을 그대로 유지되는 모습을 볼 수 있고, 필터링이 적용된 영역에서는 중앙값을 사용해 edge를 유지하면서도 노이즈를 제거하는 효과가 뚜렷하게 나타난다.

median filter에서 W는 필터의 윈도우 크기를 의미하며, 이 값을 변경하면 필터링 결과가 크게 달라진다. W의 값을 변경하며 확인한 결과 W가 작을수록 노이즈 제거 강도는 약하지만 이미지의 디테일과 윤곽선을 더 잘 보존할 수 있고, 연산량이 적어 처리 속도도 빠르다. 반면, W가 커질수록 더 넓은 범위를 참고하여 극단적인 노이즈까지 효과적으로 제거할 수 있으나, 이미지가 흐릿해지고 세부 정보가 손실되기 쉬우며 처리 속도도 느려진다. 따라서 W는 노이즈 제거 효과와 디테일 유지 사이의 균형을 조절하는 것을 확인 할 수 있었다. 위의 결과는 W가 3일 때이며, 다음은 W가 9인 경우의 실행결과이다.



그리고 다음은 Out of boundary를 zero padding으로 처리한 코드이다.

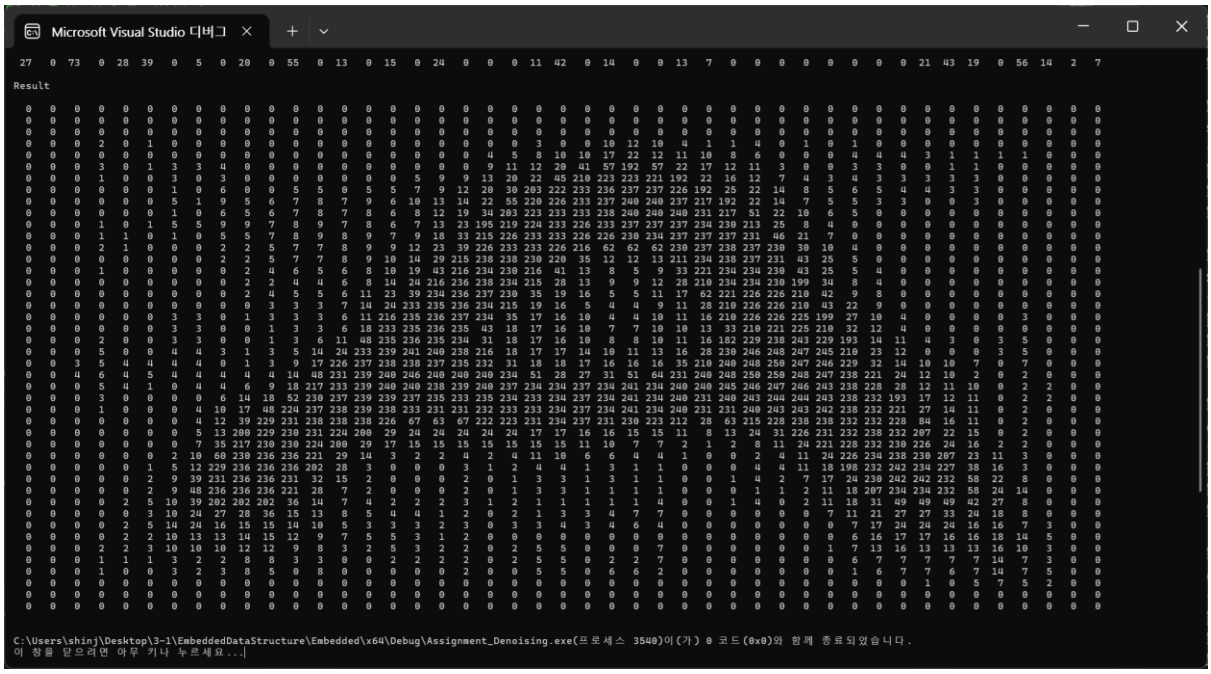
```
void median_filtering(int* input, int* output, int height, int width)
{
    int half = W / 2;
    int window[W * W];

    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {

            int k = 0;
            for (int y = -half; y <= half; y++) {
                for (int x = -half; x <= half; x++) {
                    int ny = i + y;
                    int nx = j + x;
                    if (ny < 0 || ny >= height || nx < 0 || nx >= width) {
                        window[k++] = 0; //  zero padding
                    }
                    else {
                        window[k++] = input[ny * width + nx];
                    }
                }
            }

            insertion_sort(window, W * W);
            output[i * width + j] = window[W * W / 2];
        }
    }
}
```

zero padding 방식의 median filtering 코드는 필터 윈도우가 이미지 경계를 벗어나는 경우, 그 바깥 영역을 모두 0으로 채워 넣는 방식으로 처리된다. 중심 픽셀을 기준으로 W×W 크기의 윈도우를 형성할 때, 인덱스가 이미지 바깥으로 나가면 해당 위치의 값을 0으로 간주하고 window 배열에 삽입하며, 윈도우 내 값을 모두 채운 뒤 insertion_sort()를 통해 중앙값을 계산하고 출력 배열에 대입한다. 아래는 zero padding 시 결과이다.



이번 실습을 통해 median filter의 원리와 구현 과정을 직접 경험하면서, 단순한 정렬 알고리즘을 이용해 이미지의 노이즈를 효과적으로 제거할 수 있다는 점을 알게 되었다. 특히, W×W 윈도우를 구성하고 그 안의 값들을 정렬하여 중앙값을 선택하는 방식이 salt-and-pepper noise와 같은 극단값 제거에 사용할 수 있음을 확인하였으며, 필터 크기(W)에 따라 디테일 보존과 노이즈 제거 간의 균형이 달라진다는 점도 실행해보며 알 수 있었다. out-of-boundary 상황을 처리하는 다른 방법들도 구현해보면 좋을 것 같다.